

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	PLATAFORMAS EMERGENTES (e)				
TÍTULO DE LA PRÁCTICA:	COMPARATIVA DESARROLLO DE APLICACIÓN SIN CÓDIGO				
NÚMERO DE PRÁCTICA:	1	AÑO LECTIVO:	5	NRO. SEMESTRE:	X
FECHA DE PRESENTACIÓN	06/07/2026	HORA DE PRESENTACIÓN	23/59/59		
INTEGRANTE (s) <i>Gabriel Alberto Bernedo Kaseng</i> <i>Willy Alexander Huisa Perez</i>				NOTA (0-20)	<i>Nota colocada por el docente</i>
DOCENTE(s): MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.					

RESULTADOS Y PRUEBAS
I. PROBLEMAS PROPUESTOS RESUELTOS: Problema - Se les ha pedido desarrollar una aplicación para la matrícula de estudiantes en los laboratorios (Consideren que funcionalidades les sería útil según los problemas que ustedes tienen al realizar sus matrículas). - Pero debido al poco plazo de tiempo han optado por una metodología de desarrollo rápido de aplicaciones, pero deben escoger qué herramienta de programación sin código se adaptaría mejor al programa a desarrollar. 1. Introducción a RAD El Desarrollo Rápido de Aplicaciones (RAD) es un enfoque metodológico —no una herramienta en sí— que prioriza ciclos cortos de prototipado y retroalimentación constante del

usuario por encima de la documentación exhaustiva previa, propia de modelos como el de cascada [1]. Tanto las plataformas *low-code* como las *no-code* se consideran implementaciones concretas de este enfoque RAD [1].

El *low-code* es un enfoque de desarrollo visual que permite a desarrolladores profesionales y no profesionales colaborar en la construcción de aplicaciones mediante abstracciones de alto nivel (modelos, arrastrar-y-soltar), sin eliminar por completo la posibilidad de escribir código para lógica personalizada [3], [4]. Según Gartner, para 2024 más del 65 % de la actividad de desarrollo de software correspondería a plataformas *low-code* [4].

El *no-code*, por su parte, suele tratarse como un subconjunto del enfoque *low-code*, orientado a usuarios sin conocimientos de programación ("ciudadanos desarrolladores"), que construyen aplicaciones funcionales ensamblando componentes visuales preconstruidos (formularios, tablas, condicionales tipo "si-entonces") [1], [2].

La diferencia principal entre ambos enfoques no es solo técnica sino también de público objetivo: el *low-code* está pensado para desarrolladores que buscan acelerar procesos repetitivos, mientras que el *no-code* está pensado para usuarios de negocio sin experiencia técnica [2]. Esto genera un espectro:

Desarrollo tradicional (código) → Low-Code → No-Code

A menor código, mayor velocidad de entrega y menor barrera de entrada, pero también menor capacidad de personalización de lógica compleja y mayor dependencia del proveedor (vendor lock-in) [2]. Este trade-off es el eje central que va a justificar la elección de herramienta en la sección 4.

2. Estado del Arte - Herramientas para RAD

Para el trabajo se hizo una comparación entre diferentes herramientas Low-Code / No-Code Integrada con IA :

Esta tabla se hizo en base a las referencias de la 20 a la 23.

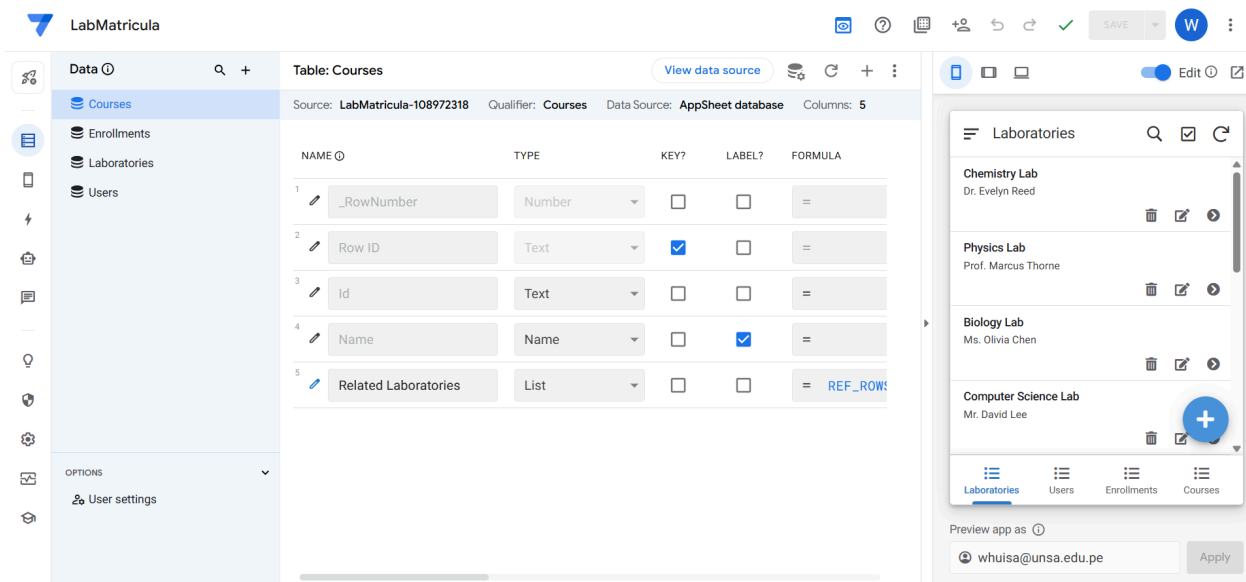
Criterio	AppSheet	FlutterFlow	Glide	Adalo
Empresa / Origen	Google	FlutterFlow Inc.	Glide Apps	Adalo
Tecnología base	Google Cloud + hojas de	Flutter (Dart) — código real	Google Sheets/Airtable/	Motor propio no-code con BD

	cálculo/BD	exportable	Glide Tables	Postgres
Tipo de app generada	PWA / híbrida (instalable)	Nativa real (Android/iOS/Web)	PWA (instalable en pantalla de inicio)	Nativa (compilada vía servicio propio)
Enfoque de generación	Parte de los datos (estructura primero)	Parte de la interfaz (pantallas primero)	Parte de los datos (hoja → app)	Parte de componentes visuales
IA generativa	Sí (Gemini/Duet AI) genera automatizaciones y estructura	Sí ("Imagine"/AI Gen) genera pantallas completas desde texto	Sí (Glide AI) genera apps y columnas con IA	Básica ("Ada") más limitada, poco robusta para apps completas
Curva de aprendizaje	Baja-Media (fácil si conoces Excel/Sheets)	Media-Alta (conceptos de widgets/estados)	Muy baja (la más rápida de prototipar)	Baja-Media (visual e intuitivo)
Personalización de UI/UX	Limitada (plantillas fijas)	Alta (control total tipo diseño)	Media (componentes predefinidos)	Media-Alta (buen editor visual)
Manejo de roles (estudiante/docente)	Bueno (seguridad por fila/usuario)	Bueno (requiere configurar auth manualmente)	Limitado	Muy bueno (permisos nativos claros)
Grilla de horario tipo calendario (según nuestra prueba)	Difícil de replicar exacto; mejor con tablas/vistas	Se acerca bien al diseño visual, pero la lógica de selección múltiple requiere trabajo manual	Muy limitado, componentes rígidos	Requiere plugins/componentes externos
Soporte offline	Sí (nativo)	Parcial (requiere configuración)	Limitado	No nativo
Base de datos / Backend	Sheets, Excel, Cloud SQL, Salesforce	Firebase, Supabase, APIs REST	Google Sheets, Airtable, Glide Tables	Postgres propio, integración con Xano

Escalabilidad	Media-Alta (empresarial en plan superior)	Alta (código exportable, producción real)	Baja-Media (pensado para MVPs)	Media
Planes de precio	Starter \$5, Core \$10, Enterprise Plus \$20 (por usuario/mes) + gratis hasta 10 usuarios de prueba	Free \$0, Basic \$39, Growth desde \$80, Business desde \$150 (por mes)	Free \$0, Basic \$25, Plus \$50, Pro \$125 (por mes)	Free \$0, Starter \$36, Professional \$52, Team \$160 (por mes, facturación anual)
Costo para proyecto académico pequeño	Muy accesible (gratis para pruebas/10 usuarios)	Accesible en Free, pero limitado (2 proyectos, 5 usos de IA)	Accesible en Free	Accesible en Free (500 registros)

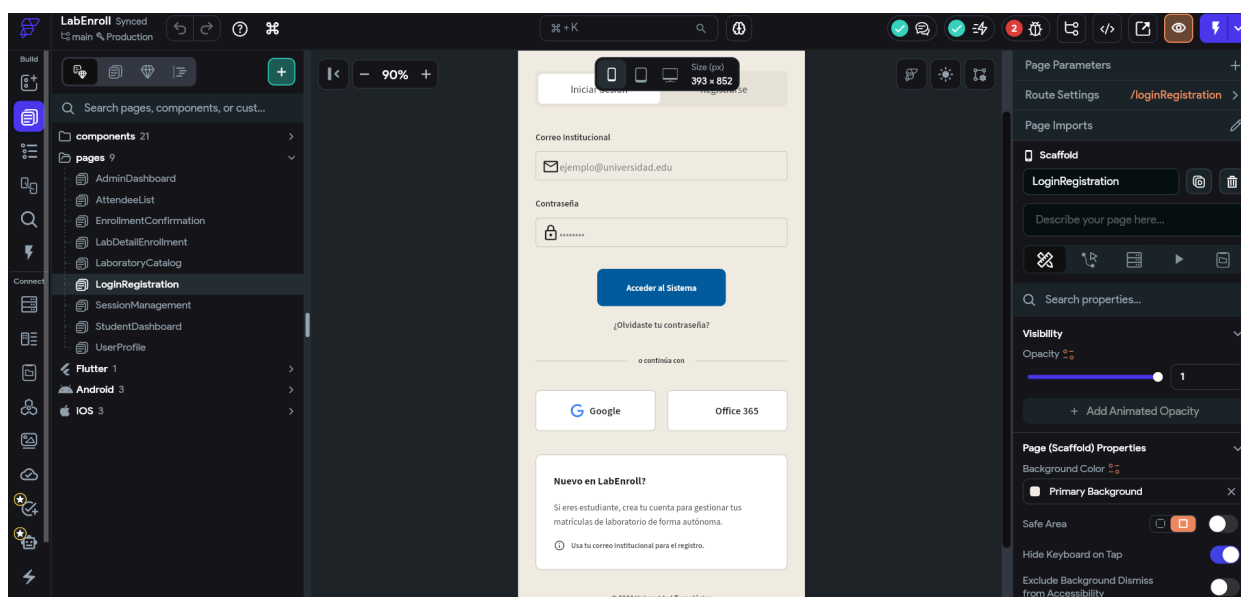
2.1 AppSheet

Google AppSheet es una plataforma no-code de Google, integrada de forma nativa a Google Workspace, que permite generar aplicaciones funcionales a partir de una fuente de datos existente (principalmente Google Sheets, aunque también Excel o bases en la nube) sin necesidad de escribir código [24]. A partir de una hoja de cálculo, AppSheet infiere automáticamente una estructura de app (formularios, vistas, listas) que el usuario puede personalizar visualmente [24]. Incluye automatizaciones nativas (envío de correos, notificaciones push, acciones condicionales), soporte para trabajo offline y funciones de captura de datos como escaneo de códigos de barras y firma digital [25]. Al pertenecer al ecosistema Google, suele venir incluido o a muy bajo costo para instituciones que ya usan Google Workspace, lo que la hace especialmente accesible para proyectos académicos [26].

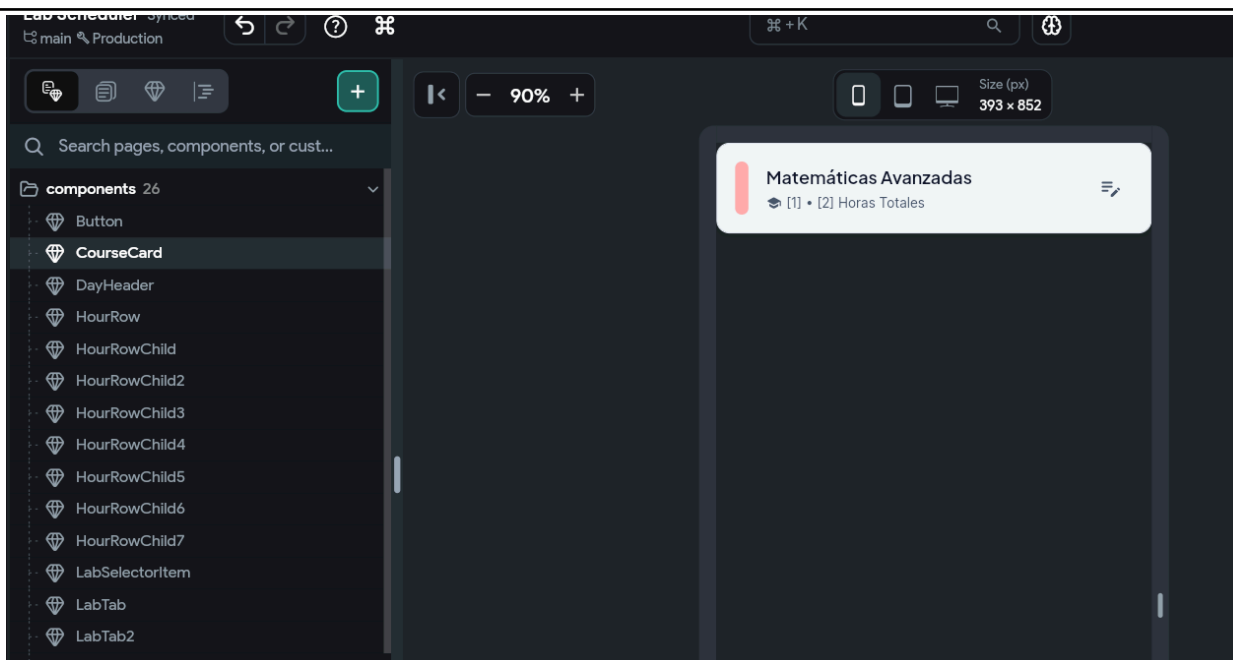


2.2 FlutterFlow

Es una plataforma low-code (no estrictamente *no-code*) que permite diseñar interfaces mediante arrastrar-y-soltar y genera código Flutter/Dart real y exportable, publicable de forma nativa en iOS, Android y web [11], [13]. Se conecta a bases de datos como Firebase o Supabase, y permite insertar funciones y lógica personalizada en Dart cuando la interfaz visual no alcanza [11], [13]. Por diseñarse sobre Flutter, requiere cierta comprensión de conceptos de programación (widgets, contenedores) para aprovecharla al máximo [13].

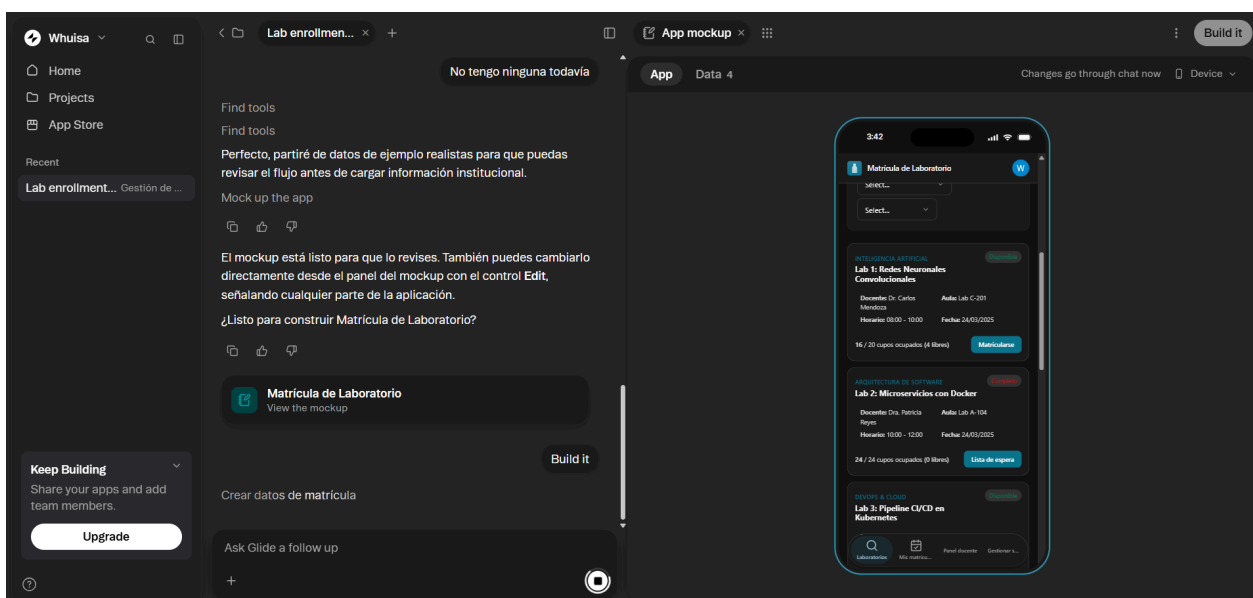


	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6



2.3 Glide

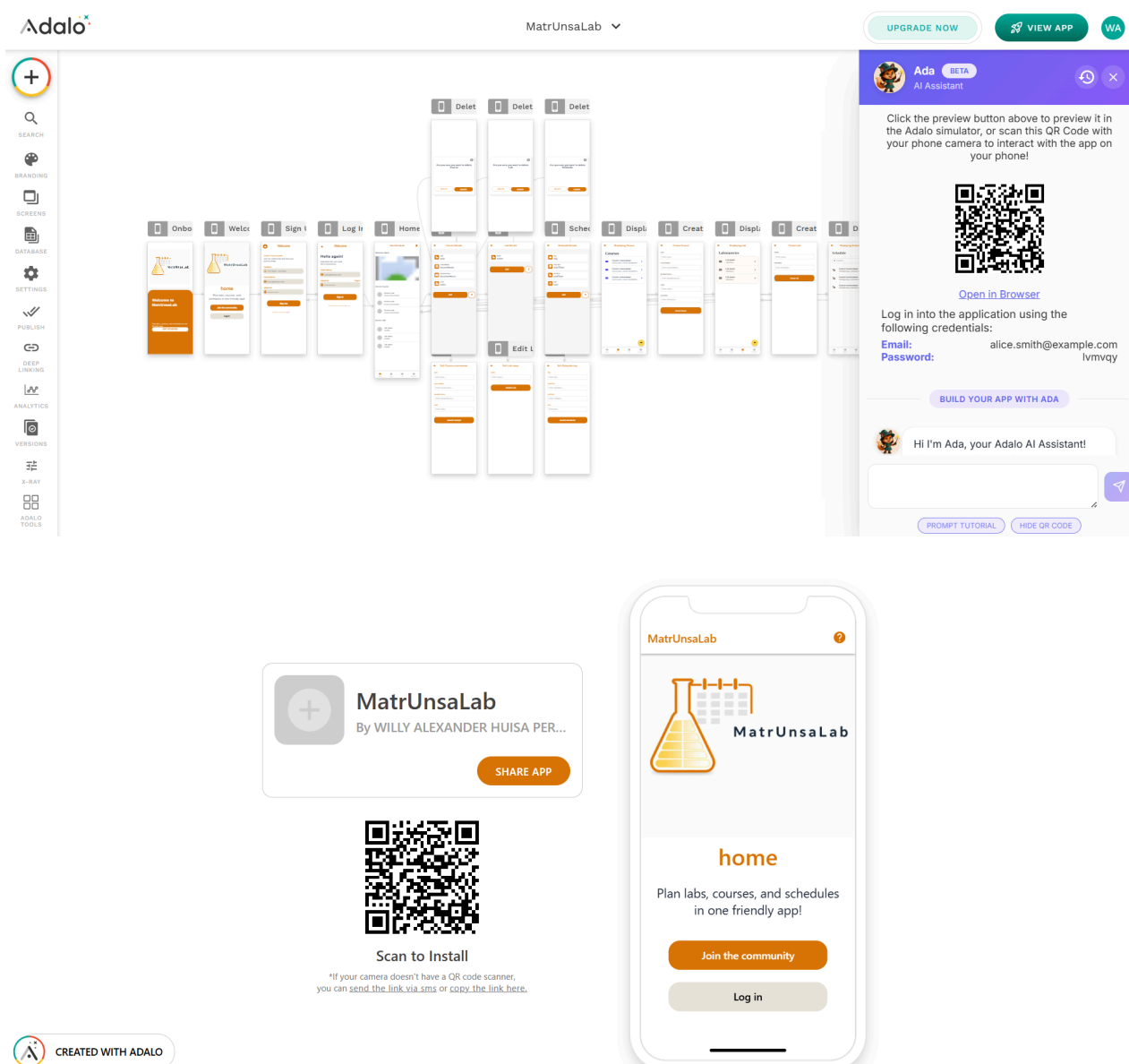
Es una plataforma no-code que convierte hojas de cálculo (Google Sheets, Airtable) o bases SQL en aplicaciones web funcionales de forma casi automática [16]. Está orientada a herramientas internas y prototipos de datos tabulares, con una curva de aprendizaje muy baja, pero con capacidades limitadas para lógica de negocio compleja [16].



	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

2.4 Adalo

Es una plataforma no-code genuina, orientada a la creación de aplicaciones móviles y web mediante un editor visual tipo "PowerPoint", con base de datos propia incluida (sin límite de registros en planes pagos) y publicación directa en App Store y Google Play [14], [15]. Permite lógica condicional básica mediante "Acciones" configurables visualmente, pero no permite escribir ni exportar código [11], [15].



The image displays the Adalo platform interface for creating a mobile application named "MatrUnsaLab". The main workspace shows a visual flowchart of the app's screens, including "Onbo", "Welcr", "Sign i", "Log It", "Home", "Delet", "Schee", "Displ", "Creat", and "Edit L". A sidebar on the left contains various tool icons like "SEARCH", "BRANDING", "SCREENS", "DATABASE", "SETTINGS", "PUBLISH", "DEEP LINKING", "ANALYTICS", "VERSIONS", "X-RAY", and "ADALO TOOLS". On the right, there's a preview window showing the app's login screen with a QR code and login fields. Below the main workspace, there's a section for sharing the app, including a QR code labeled "Scan to Install" and a "SHARE APP" button. At the bottom left, it says "CREATED WITH ADALO".

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

3. Diseño del Sistema

3.1 Actores del sistema

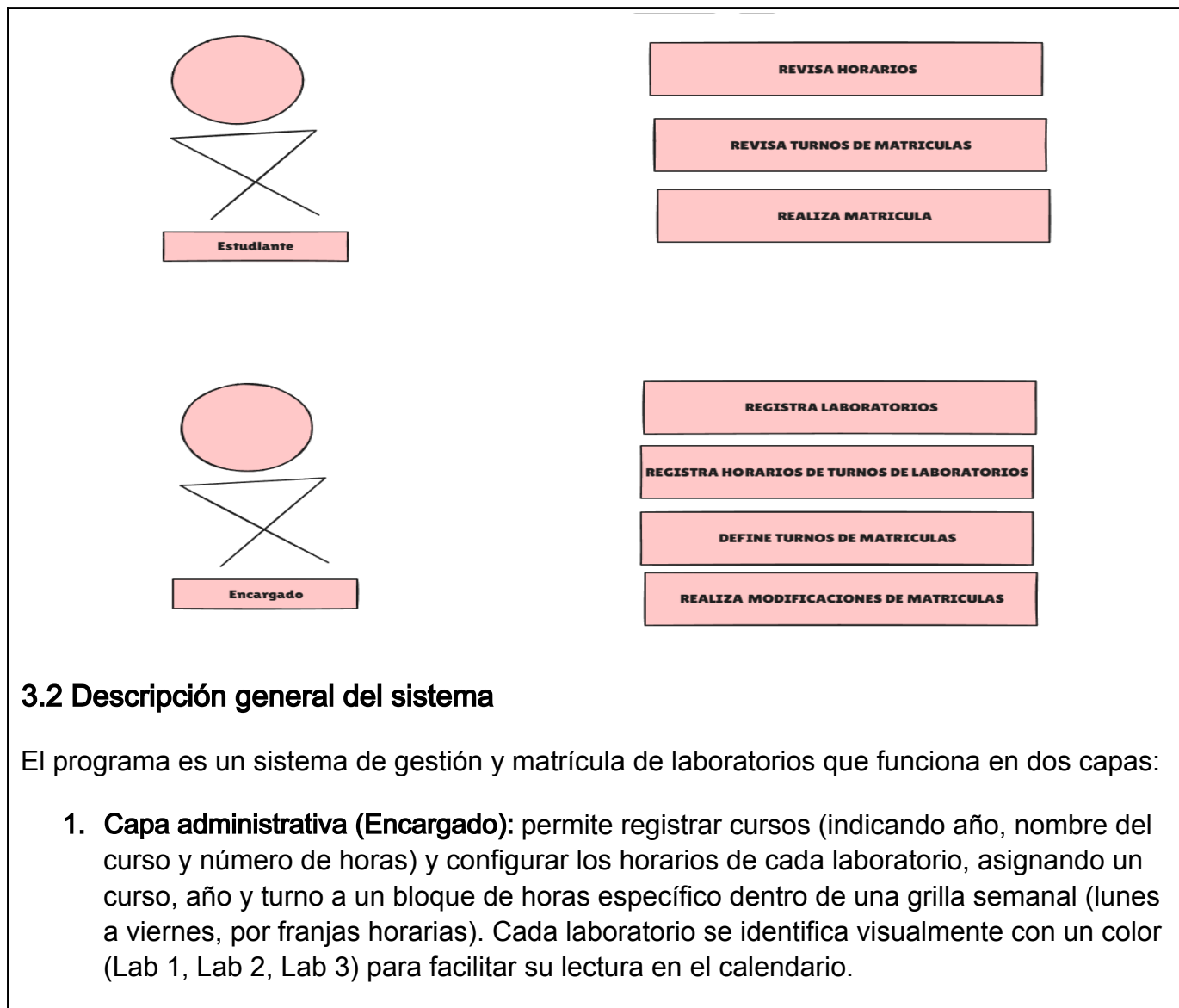
El sistema contempla dos actores principales, cada uno con responsabilidades bien diferenciadas:

Estudiante

- Revisa los horarios disponibles de los laboratorios.
- Revisa los turnos de matrícula habilitados (fechas/horas en las que le corresponde matricularse).
- Realiza su matrícula seleccionando los cursos y generando un horario.

Encargado (administrador del sistema / coordinador de laboratorios)

- Registra los laboratorios existentes (Lab 1, Lab 2, Lab 3, etc.).
- Registra los horarios de turnos de cada laboratorio (qué días y horas están disponibles).
- Define los turnos de matrícula (a qué grupo de estudiantes le toca matricularse y cuándo).
- Realiza modificaciones sobre matrículas ya registradas (correcciones, reasignaciones, etc.).



3.2 Descripción general del sistema

El programa es un sistema de gestión y matrícula de laboratorios que funciona en dos capas:

1. **Capa administrativa (Encargado):** permite registrar cursos (indicando año, nombre del curso y número de horas) y configurar los horarios de cada laboratorio, asignando un curso, año y turno a un bloque de horas específico dentro de una grilla semanal (lunes a viernes, por franjas horarias). Cada laboratorio se identifica visualmente con un color (Lab 1, Lab 2, Lab 3) para facilitar su lectura en el calendario.

LAB 1
LAB 2
LAB 3

HORA	LUNES	MARTES	MIÉRCOLES	JUEVES	VIERNES
07:00	TRABAJO DE INVEST. LABORATORIO A	DESARROLLO DE SOL. LABORATORIO A	TRABAJO DE INVEST. LABORATORIO A	PROYECTO DE INGEN. LABORATORIO A	PROYECTO DE INGEN. LABORATORIO A
08:00					
09:00	PRACTICAS PRO. INGEN. LABORATORIO A	DESARROLLO DE SOL. LABORATORIO A	PRACTICAS PRO. INGEN. LABORATORIO A	PRACTICAS PRO. INGEN. LABORATORIO A	PRACTICAS PRO. INGEN. LABORATORIO A
10:00					
11:00	PRACTICAS PRO. INGEN. LABORATORIO A	DESARROLLO DE SOL. LABORATORIO A	PRACTICAS PRO. INGEN. LABORATORIO A	PRACTICAS PRO. INGEN. LABORATORIO A	PRACTICAS PRO. INGEN. LABORATORIO A
12:00					
13:00	PRACTICAS PRO. INGEN. LABORATORIO A	DESARROLLO DE SOL. LABORATORIO A	PRACTICAS PRO. INGEN. LABORATORIO A	PRACTICAS PRO. INGEN. LABORATORIO A	PRACTICAS PRO. INGEN. LABORATORIO A
14:00					
15:00	PRACTICAS PRO. INGEN. LABORATORIO A	DESARROLLO DE SOL. LABORATORIO A	PRACTICAS PRO. INGEN. LABORATORIO A	PRACTICAS PRO. INGEN. LABORATORIO A	PRACTICAS PRO. INGEN. LABORATORIO A
16:00					

Registrar Curso REGISTRAR

Año 5to HORAS - 3 +

Nombre del Curso Temas Avanzados en Ingeniería de Software

Nuevo Horario REGISTRAR / MODIFICAR

Año 5to Turno C

Curso Temas Avanzados en Ingeniería de Software

Selecione las horas en el GRID

2. **Capa de matrícula (Estudiante):** el estudiante visualiza la oferta de laboratorios organizada por hoja/vista (por ejemplo "Laboratorio - Hoja 1", "Teoría - Hoja 1"), filtra los cursos que le interesan mediante un panel de selección con checkboxes (curso, sección, tipo: teoría o práctica), y el sistema genera automáticamente una propuesta de horario que combina los cursos elegidos sin cruces de horario. Si existe más de una combinación posible sin conflictos, el sistema muestra varias opciones navegables ("Opción 1 de 1", "Opción 2 de 1", etc.), y el estudiante confirma su matrícula con el botón "MATRICULAR!!".

ORDENAR

 DÍA LIBRE: L M M J V
 RANGO HORARIO: 07:00 - 20:30

CURSOS
☒ Trabajo de Investigación
 TEORÍA
 Sec. A ☐ Sec. B ☐ Sec. C ☐
 Sec. D ☐ Sec. E ☐ Sec. F ☐
 Sec. G ☐ Sec. H ☐
☒ Practicas Pre Profesionales
 TEORÍA
 Sec. A ☐ Sec. B ☐
☒ Proyecto de Ingeniería de Software 2
 TEORÍA
 Sec. A ☐ Sec. B ☐
 LABORATORIO
 Sec. A ☐ Sec. B ☐ Sec. C ☐

HORARIO GENERADO

MATRICULAR !!

← OPCIÓN 1 DE 1 →

HORA	LUNES	MARTES	MIÉRCOLES	JUEVES	VIERNES
07:00	TRABAJO DE INVEST. TEORÍA A 07:00 - 09:40	DESARROLLO DE SOF. LABORATORIO A 07:00 - 09:40	TRABAJO DE INVEST. TEORÍA A 07:00 - 09:40		PROYECTO DE INGEN. LABORATORIO A 07:00 - 09:40
08:00					
09:00					
10:00	PRÁCTICAS PRE PROF. TEORÍA A 09:40 - 11:30		DESARROLLO DE SOF. TEORÍA A 09:00 - 11:30	PLATAFORMAS ERIEOL. TEORÍA A 09:00 - 11:30	PRÁCTICAS PRE PROF. TEORÍA A 09:00 - 11:30
11:00					
12:00				TOPICOS AVANZADOS. TEORÍA A 11:30 - 14:00	PROYECTO DE INGEN. TEORÍA A 11:30 - 13:30
13:00					
14:00					
15:00					
16:00					

VISTAS DISPONIBLES

TEORÍA

HOJA 1

HOJA 2

LABORATORIO

HOJA 1 →

HOJA 2

LABORATORIO - HOJA 1

12 CLASES

HORA	LUNES	MARTES	MIÉRCOLES	JUEVES	VIERNES
07:00	TOPICOS AVANZADOS. LABORATORIO A 07:00 - 09:40	DESARROLLO DE SOF. LABORATORIO A 07:00 - 09:40		DESARROLLO DE SOF. LABORATORIO C 07:00 - 09:40	PROYECTO DE INGEN. LABORATORIO A 07:00 - 09:40
08:00					
09:00	TOPICOS AVANZADOS. LABORATORIO B 09:00 - 11:30				PROYECTO DE INGEN. LABORATORIO C 09:00 - 11:30
10:00					
11:00					
12:00					
13:00					
14:00	PROYECTO DE INGEN. LABORATORIO D 14:00 - 15:40		PROYECTO DE INGEN. LABORATORIO B 14:00 - 15:40		
15:00					
16:00				TOPICOS AVANZADOS. LABORATORIO C 16:00 - 18:30	

3.3 Funcionalidades principales

Módulo	Funcionalidad	Actor
Gestión de cursos	Registrar curso (año, nombre, número de horas)	Encargado
Gestión de laboratorios	Registrar laboratorio y asignarle un identificador/color	Encargado

Gestión de horarios	Registrar/modificar horario de un laboratorio seleccionando bloques en una grilla semanal	Encargado
Gestión de turnos	Definir turnos de matrícula por año/grupo	Encargado
Modificación de matrículas	Editar o corregir una matrícula ya realizada	Encargado
Consulta de horarios	Visualizar horarios disponibles por laboratorio o por curso (vista "Hoja 1", "Hoja 2", etc.)	Estudiante
Consulta de turno	Revisar en qué turno/fecha le corresponde matricularse	Estudiante
Selección de cursos	Filtrar cursos y secciones de interés mediante checkboxes	Estudiante
Generación automática de horario	Combinar los cursos seleccionados evitando cruces, mostrando una o varias opciones válidas	Estudiante
Confirmación de matrícula	Confirmar la opción de horario elegida mediante el botón "Matricular"	Estudiante

• Luego deben agregar una sección donde seleccionen una herramienta para la solución del problema e indiquen las razones por las cuales escogieron dicha herramienta según la aplicación planteada.

• Finalmente deberán brindar unas conclusiones de su análisis donde señalen que han podido descubrir al hacer su comparación o al seleccionar la herramienta.

4. Resultados de la Comparativa de Herramientas

El estado del arte actual referente a las herramientas No-Code / Low-Code es bastante amplio, a pesar de que en esta ocasión se han considerado solo 4, podemos considerar los siguientes criterios para elegir la herramienta adecuada:

- **Facilidad de Uso:** Glide y AppSheet tienen la curva de aprendizaje más baja de las cuatro, porque parten de una hoja de cálculo y su lógica se parece bastante a Excel. Adalo también resulta intuitiva gracias a su

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13

editor de arrastrar y soltar. FlutterFlow pide un poco más de esfuerzo al inicio: hay que entender conceptos propios del desarrollo de apps como widgets, estados y acciones. Ese esfuerzo extra se compensa con un editor visual bien pensado y un asistente de IA que ayuda a no perderse en el proceso.

- **Eficiencia:** Conviene separar dos cosas: qué tan rápido se construye la app y qué tan bien corre una vez terminada. En velocidad de construcción, Glide y AppSheet ganan fácil, se puede tener un prototipo funcionando en minutos. En rendimiento final la cosa cambia: FlutterFlow compila a código Flutter/Dart real, así que las apps resultantes se sienten más fluidas y aprovechan mejor los recursos del celular que las PWA de Glide o AppSheet.
- **Nivel de Utilidad:** Para el problema de matrícula planteado (cupos, cruces de horario, roles, una grilla tipo calendario), FlutterFlow es la que más da de sí, porque el control total sobre interfaz y lógica permite implementar reglas de negocio complicadas sin chocar con las plantillas rígidas de Glide o AppSheet. Adalo maneja bien los roles, pero se queda corta con la interfaz de calendario si no se recurre a componentes externos.
- **Integración con IA:** Las cuatro herramientas traen algún asistente de IA, aunque no todas con el mismo nivel de madurez. FlutterFlow tiene una función para generar pantallas completas a partir de un prompt de texto (Imagine) y un asistente conversacional que modifica lógica y código directamente. AppSheet usa la IA más que nada para automatizaciones, Glide para armar estructuras de datos, y Adalo (Ada) se queda en sugerencias puntuales, todavía bastante básicas.
- **Modificabilidad:** Acá FlutterFlow saca ventaja clara: genera código Flutter real y exportable, permite meter código Dart propio y seguir trabajando el proyecto incluso fuera de la plataforma. Las otras tres dejan al desarrollador encerrado en su propio ecosistema, lo cual limita bastante las modificaciones profundas cuando el proyecto crece más de lo que el editor visual permite.
- **Instalabilidad:** FlutterFlow y Adalo generan apps nativas de verdad, instalables como APK o IPA y publicables en Play Store y App Store. AppSheet y Glide, en cambio, producen PWA, que se instalan desde el navegador sin pasar por una tienda oficial. Si el requisito es "aplicación móvil" en sentido estricto, la instalabilidad nativa de FlutterFlow pesa a favor frente a las opciones basadas en PWA.
- **Estabilidad:** Al compilar a código nativo, las apps de FlutterFlow no dependen todo el tiempo de un servidor intermedio para mostrar la interfaz, algo que sí ocurre con las PWA, que necesitan buena conexión con la plataforma para funcionar bien. Sumado a eso, Flutter es un framework respaldado por Google con una comunidad grande y activa, lo que da cierta garantía de soporte y mantenimiento a largo plazo y refuerza esta elección final.

De esta forma, la herramienta que vemos más eficiente para RAD, en esta ocasión rondaría entre "FlutterFlow"

5. ANEXO

REPOSITORIO DE GITHUB:

https://github.com/Alex05ander2004/PlataformasEmergentes_Labs/tree/main/LAB01

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 14

CONCLUSIONES

Las plataformas Low-Code y No-Code constituyen una implementación práctica y sumamente útil del enfoque de Desarrollo Rápido de Aplicaciones (RAD). Estas herramientas permiten ciclos de prototipado mucho más ágiles que el modelo de desarrollo tradicional, facilitando una respuesta rápida a las necesidades del usuario final.

La selección de la herramienta adecuada es un factor crítico que debe basarse en los requisitos específicos de cada proyecto. Es fundamental encontrar un equilibrio entre la facilidad de uso inicial, el grado de personalización visual y lógica requerido, y las capacidades de escalabilidad a largo plazo de la solución.

Para este caso de estudio, se identifica a FlutterFlow como una herramienta eficiente. A pesar de presentar una curva de aprendizaje más pronunciada en comparación con Glide o Adalo, su capacidad para ofrecer un control superior sobre la lógica de negocio y un diseño nativo exportable justifica su elección frente a opciones puramente No-Code.

En conclusión, la integración de la Inteligencia Artificial generativa en estas plataformas está democratizando el desarrollo de software. Esta transformación mejora significativamente la accesibilidad, permitiendo que personas con menor experiencia técnica generen soluciones funcionales y robustas en tiempos récord.

METODOLOGÍA DE TRABAJO

Colocar la metodología de trabajo que ha utilizado el estudiante o el grupo para resolver la práctica, es decir el procedimiento/secuencia de pasos en forma general.

REFERENCIAS Y BIBLIOGRAFÍA

- [1] IBM, "Low-Code vs. No-Code: What's the Difference?" 2026. [Online]. Available: <https://www.ibm.com/think/topics/low-code-vs-no-code>. [Accessed: 1-Sep-2026].
- [2] IBM, "What Is No Code?" 2026. [Online]. Available: <https://www.ibm.com/think/topics/no-code>. [Accessed: 1-Sep-2026].
- [3] SAP, "What is Low-Code/No-Code?" 2026. [Online]. Available: <https://www.sap.com/india/products/technology-platform/build/what-is-low-code-no-code.html>. [Accessed: 1-Sep-2026].
- [4] U.S. Patent and Trademark Office, "System and method for automated software development through AI based automated requirements," patent filing citing Gartner's definition of low-code application platforms. [Online]. Available: <https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/12265922>. [Accessed: 1-Sep-2026].
- [5] SoftwareSuggest, "Bubble Pricing (2026): Explore Plans, Features & Key Insights." [Online]. Available: <https://www.softwaresuggest.com/bubble/pricing>. [Accessed: 1-Sep-2026].
- [6] ProductGrowth, "Bubble.io No-Code App Builder & Pricing." [Online]. Available: <https://productgrowth.in/tools/no-code/bubble/>. [Accessed: 1-Sep-2026].
- [7] AlphaBold, "Power Apps Pricing: Plans, Costs & 2026 Guide." [Online]. Available: <https://www.alphabold.com/power-apps-pricing/>. [Accessed: 1-Sep-2026].

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 15

- [8] CostBench, "Glide Pricing 2026: 3 Plans from Free-\$199/month." [Online]. Available: <https://costbench.com/software/no-code/glide/>. [Accessed: 1-Sep-2026].
- [9] CheckThat.ai, "AppSheet Pricing 2026: 3 Plans from \$5-\$20/user/month." [Online]. Available: <https://checkthat.ai/brands/appsheet/pricing>. [Accessed: 1-Sep-2026].
- [10] Softabase, "AppSheet Review 2026 - Features, Pricing & Pros/Cons." [Online]. Available: <https://softabase.com/software/no-code-low-code/appsheet>. [Accessed: 1-Sep-2026].
- [11] Adalo, "FlutterFlow Pricing 2026: \$80/mo Per Seat + Hidden Database Costs." [Online]. Available: <https://www.adalo.com/posts/flutterflow-pricing/>. [Accessed: 1-Sep-2026].
- [12] NoCode MBA, "FlutterFlow Pricing 2026: Every Plan Explained." [Online]. Available: <https://www.nocode.mba/articles/flutterflow-pricing>. [Accessed: 1-Sep-2026].
- [13] SuperApp, "FlutterFlow Review (2026): Pricing, Real Reviews, Is It Worth It?" [Online]. Available: <https://www.superapp.com/blog/flutterflow-review-2026-pricing-real-reviews-is-it-worth-it>. [Accessed: 1-Sep-2026].
- [14] Adalo, "The 2026 Pricing Guide for No Code Mobile App Maker Platforms." [Online]. Available: <https://www.adalo.com/posts/pricing-guide-no-code-mobile-app-maker/>. [Accessed: 1-Sep-2026].
- [15] Cybernews, "Adalo review 2026: features, pros and cons, pricing, and who it is for." [Online]. Available: <https://cybernews.com/ai-tools/adalo-review/>. [Accessed: 1-Sep-2026].
- [16] Cybernews, "Glide review 2026: complete hands-on evaluation of no-code app builder." [Online]. Available: <https://cybernews.com/ai-tools/glide-review/>. [Accessed: 1-Sep-2026].
- [17] Google Developers, "Google Apps Script overview." [Online]. Available: <https://developers.google.com/apps-script/overview>. [Accessed: 1-Sep-2026].
- [18] Kissflow, "What Is a Google Apps Script?" [Online]. Available: <https://kissflow.com/workflow/google-apps/what-is-google-apps-script/>. [Accessed: 1-Sep-2026].
- [19] AutomationAtlas, "Google Apps Script: Free Automation for Google Workspace." [Online]. Available: <https://automationatlas.io/tools/google-apps-script/>. [Accessed: 1-Sep-2026].
- [20] Google, "AppSheet Pricing," 2026. [Online]. Available: <https://about.appsheet.com/pricing/>. [Accessed: 1-Sep-2026].
- [21] FlutterFlow, "Pricing," 2026. [Online]. Available: <https://flutterflow.io/pricing>. [Accessed: 1-Sep-2026].
- [22] Glide, "Pricing," 2026. [Online]. Available: <https://www.glideapps.com/pricing>. [Accessed: 1-Sep-2026].
- [23] Adalo, "Pricing," 2026. [Online]. Available: <https://www.adalo.com/pricing>. [Accessed: 1-Sep-2026].
- [24] Google Developers, "AppSheet overview," Google. [Online]. Available: <https://about.appsheet.com/>. [Accessed: 1-Sep-2026].
- [25] Tekpon, "Google Cloud AppSheet Pricing Reviews for 2026." [Online]. Available: <https://tekpon.com/software/google-cloud-appsheet/pricing/>. [Accessed: 1-Sep-2026].
- [26] Softabase, "AppSheet Review 2026 - Features, Pricing & Pros/Cons." [Online]. Available: <https://softabase.com/software/no-code-low-code/appsheet>. [Accessed: 1-Sep-2026].